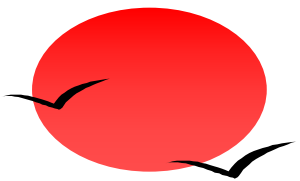


系统架构设计师

系统架构进阶 篇

第12章 系统质量属性与架构评估

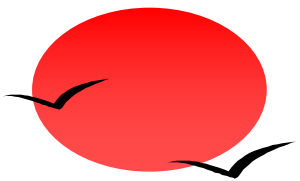
讲师：诸葛老师



本章学习建议

- 根据考试大纲，本章不仅考查系统架构设计师**单选题**，预计考**10分**左右，而且**案例分析和论文写作**也是**必考**，对应第二版教材第8章，属于重点学习的章节。





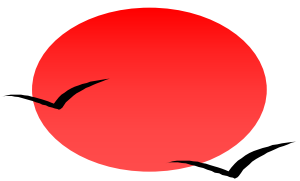
12.1 软件系统质量属性-12.1.1 质量属性概念

- **软件系统质量属性**（Quality Attribute）是一个系统的可测量或者可测试的属性，用来描述系统满足利益相关者（Stakeholders）需求的程度。基于软件系统的生命周期，可以将**软件系统的质量属性**分为**开发期质量属性**和**运行期质量属性**2个部分。

- **1. 开发期质量属性**

开发期质量属性主要指在**软件开发阶段**所关注的质量属性，主要包含6个方面。

- （1）**易理解性**：指设计被开发人员理解的难易程度。
- （2）**可扩展性**：软件因适应新需求或需求变化而增加新功能的能力，也称为灵活性。
- （3）**可重用性**：指重用软件系统或某一部分的难易程度。
- （4）**可测试性**：对软件测试以证明其满足需求规范的难易程度。
- （5）**可维护性**：当需要修改缺陷、增加功能、提高质量属性时，识别修改点并实施修改的难易程度。
- （6）**可移植性**：将软件系统从一个运行环境转移到另一个不同的运行环境的难易程度。

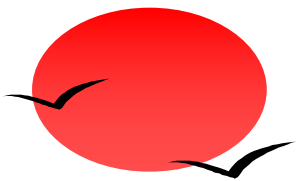


12.1.1 质量属性概念

■ 2. 运行期质量属性

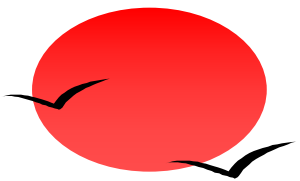
运行期质量属性主要指在**软件运行阶段**所关注的质量属性，主要包含7个方面。

- (1) **性能**：性能是指软件系统及时提供相应服务的能力，如速度、吞吐量和容量等的要求。
- (2) **安全性**：指软件系统同时兼顾向合法用户提供服务，以及阻止非授权使用的能力。
- (3) **可伸缩性**：指当用户数和数据量增加时，软件系统维持高服务质量的能力。例如，通过增加服务器来提高能力。
- (4) **互操作性**：指本软件系统与其他系统交换数据和相互调用服务的难易程度。
- (5) **可靠性**：软件系统在一定的时间内持续无故障运行的能力。
- (6) **可用性**：指系统在一定时间内正常工作的时间所占的比例。可用性会受到系统错误，恶意攻击，高负载等问题的影响。
- (7) **鲁棒性**：是指软件系统在非正常情况（如用户进行了非法操作、相关的软硬件系统发生了故障等）下仍能够正常运行的能力，也称健壮性或容错性。



本节练习

- (1)基于软件系统的生命周期，可以将软件系统的质量属性分为____两个部分。
 - A.需求分析期质量属性和设计期质量属性
 - B.开发期质量属性和运行期质量属性
 - C.设计期质量属性和开发期质量属性
 - D.设计期质量属性和运行期质量属性
- 答案：B
- 解析：软件系统质量属性（Quality Attribute）是一个系统的可测量或者可测试的属性，用来描述系统满足利益相关者（Stakeholders）需求的程度。基于软件系统的生命周期，可以将软件系统的质量属性分为开发期质量属性和运行期质量属性2个部分。开发期质量属性主要指在软件开发阶段所关注的质量属性，主要包含6个方面：易理解性、可扩展性、可重用性、可测试性、可维护性、可移植性。运行期质量属性主要指在软件运行阶段所关注的质量属性，主要包含7个方面：性能、安全性、可伸缩性、互操作性、可靠性、可用性、鲁棒性。



本节练习

- (2)在软件系统质量属性(Quality Attribute)中，_____关注系统在一定时间内正常工作的时间所占的比例；_____关注软件系统与其他系统交换数据和相互调用服务的难易程度。

- | | | | |
|---------|--------|--------|-------|
| ■ A.可用性 | B.可修改性 | C.性能 | D.安全性 |
| ■ A.可靠性 | B.可伸缩性 | C.互操作性 | D.易用性 |

- 答案：A C

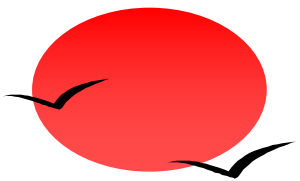
- 解析：（1）性能：性能是指软件系统及时提供相应服务的能力，如速度、吞吐量和容量等的要求。

（2）安全性：指软件系统同时兼顾向合法用户提供服务，以及阻止非授权使用的能力。

（3）可伸缩性：指当用户数和数据量增加时，软件系统维持高服务质量的能力。例如，通过增加服务器来提高能力。（4）互操作性：指本软件系统与其他系统交换数据和相互调用服务的难易程度。

（5）可靠性：软件系统在一定的时间内持续无故障运行的能力。

（6）可用性：指系统在一定时间内正常工作的时间所占的比例。可用性会受到系统错误，恶意攻击，高负载等问题的影响。



12.1.2 面向架构评估的质量属性

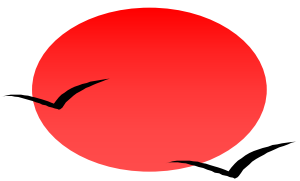
- 在架构评估过程中，所普遍关注的质量属性有以下几种。

(1) **性能**：指**系统的响应能力**，即要经过多长时间才能对某个事件做出响应，或者在某段时间内系统所能处理的事件的**个数**。如**响应时间、吞吐量**。设计策略：**优先级队列、增加计算资源、减少计算开销、引入并发机制、采用资源调度等**。

(2) **可靠性**：是软件系统在应用或系统错误面前，**在意外或错误使用的情况下维持软件系统的功能特性的基本能力**。可靠性通常用平均失效等待时间（Mean Time To Failure, **MTTF**）和平均失效间隔时间（Mean Time Between Failure, **MTBF**）来衡量。在失效率为常数和修复时间很短的情况下，**MTTF 和MTBF几乎相等**。可分为**容错和健壮性**。**容错**是指在错误发生时确保系统正确的行为，并进行内部“修复”。**健壮性**是指系统不受错误使用和错误输入的影响，按既定程序忽略错误。设计策略：**冗余、心跳、选举、Ping/Echo**。

(3) **可用性**：是系统**能够正常运行的时间比例**，经常用两次故障之间的时间长度或在出现故障时系统能够恢复正常的速度来表示。如**故障间隔时间**。设计策略：**冗余、心跳、选举、Ping/Echo**。

(4) **安全性**：是指**系统在向合法用户提供服务的同时能够阻止非授权用户使用的企图或拒绝服务的能力**。



12.1.2 面向架构评估的质量属性

可分为机密性、完整性、不可否认性、可控性。机密性保证信息不泄露给未授权的用户、实体或过程；完整性保证信息的完整和准确，防止信息被非法修改；不可否认性是指信息交换的双方不能否认其在交换过程中发送信息或接收信息的行为；可控性保证对信息的传播及内容具有控制的能力，防止为非法者所用。设计策略：入侵检测、用户认证、用户授权、追踪审计。

(5) 可修改性：指能够快速以较高的性能价格比对系统进行变更的能力。通常以某些具体的变更为基准，通过考查这些变更的代价来衡量可修改性。可分为可维护性、可扩展性、结构重组和可移植性。

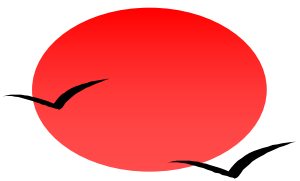
可维护性：在错误发生后“修复”软件系统的难易程度。

可扩展性：软件因适应新需求或需求变化而增加新功能的能力。

结构重组：重新组织软件系统的构件及构件之间的关系。

可移植性：将软件系统从一个运行环境转移到另一个不同的运行环境的难易程度。

设计策略：接口-实现分类、抽象、信息隐藏。

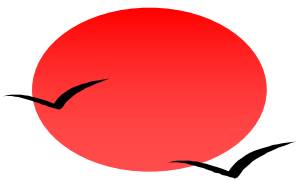


12.1.2 面向架构评估的质量属性

(6) **功能性**：是系统所能完成所期望的工作的能力。一项任务的完成需要系统中许多或大多数构件的相互协作。

(7) **可变性**：指架构经扩充或变更而成为新架构的能力。这种新架构应该符合预先定义的规则，在某些具体方面不同于原有的架构。当要将某个架构作为一系列相关产品的基础时，可变性是很重要的。

(8) **互操作性**：作为系统组成部分的软件不是独立存在的，通常与其他系统或自身环境相互作用。为了支持互操作性，软件架构必须为外部可视的功能特性和数据结构提供精心设计的软件入口。程序和用其他编程语言编写的软件系统的交互作用就是互操作性的问题，这种互操作性也影响应用的软件架构。



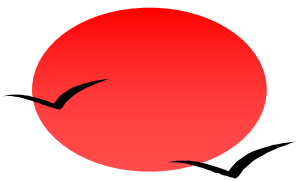
本节练习

- (3)安全性(Security)是指系统在向合法用户提供服务的同时能够阻止非授权用户使用的企图或拒绝服务的能力。安全性可根据系统可能受到的安全威胁类型来分类。其中，_____保证信息不泄露给未授权的用户、实体或过程；_____是指信息交换的双方不能否认其在交换过程中发送信息或接收信息的行为。

- | | | | |
|---------|-------|---------|-------|
| ■ A.可控性 | B.完整性 | C.不可否认性 | D.机密性 |
| ■ A.完整性 | B.机密性 | C.不可否认性 | D.可控性 |

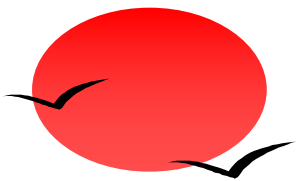
- 答案：D C

- 解析：安全性(Security)是指系统在向合法用户提供服务的同时能够阻止非授权用户使用的企图或拒绝服务的能力。安全性可根据系统可能受到的安全威胁类型来分类。其中，**机密性**保证信息不泄露给未授权的用户、实体或过程；**不可否认性**是指信息交换的双方不能否认其在交换过程中发送信息或接收信息的行为。



本节练习

- (4)平均失效等待时间（mean time to failure, MTTF）和平均失效间隔时间（mean time between failure, MTBF）是进行系统可靠性分析时的重要指标，在失效率为常数和修复时间很短的情况下， ____。
 - A.MTTF远远小于MTBF
 - B.MTTF和MTBF无法计算
 - C.MTTF远远大于MTBF
 - D.MTTF和MTBF几乎相等
- 答案： D
- 解析： 可靠性通常用平均失效等待时间（Mean Time To Failure, **MTTF**）和平均失效间隔时间（Mean Time Between Failure, **MTBF**）来衡量。在失效率为常数和修复时间很短的情况下，**MTTF 和MTBF几乎相等**。



12.1.3 质量属性场景描述

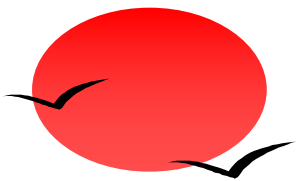
- 为了精确描述软件系统的质量属性，通常采用**质量属性场景**（Quality Attribute Scenario）作为描述质量属性的手段。质量属性场景是一种**面向特定质量属性的需求**。它由6部分组成：
 - 刺激源（Source）：这是某个**生成该刺激的实体**（人、计算机系统或者任何其他刺激器）。
 - 刺激（Stimulus）：该刺激是**当刺激到达系统时需要考虑的条件**。
 - 环境（Environment）：该刺激**在某些条件内发生**。当激励发生时，系统可能处于过载、运行或者其他情况。
 - 制品（Artifact）：某个制品**被激励**。这可能是整个系统，也可能是系统的一部分。
 - 响应（Response）：该响应是**在激励到达后所采取的行动**。
 - 响应度量（Measurement）：当响应发生时，**应当能够以某种方式对其进行度量**，以对需求进行测试。

可修改性质量属性场景描述实例（以淘宝APP为例）：

刺激源：淘宝APP进行更新，开发人员修改代码；刺激：淘宝APP暂时维护，增加部分功能；

环境：APP平台；制品：新版本的淘宝APP；响应：增添部分功能或修复部分BUG；

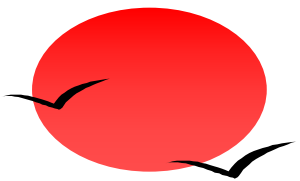
响应度量：过去的BUG消失或增加了新功能。



12.1.3 质量属性场景描述

■ 可修改性质量属性场景描述实例：

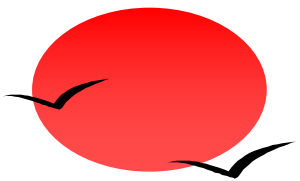
场景要素	可能的情况
刺激源	最终用户、开发人员、系统管理员
刺激	希望增加、删除、修改、改变功能、质量属性、容量等
环境	系统设计时、编译时、构建时、运行时
制品	系统用户界面、平台、环境或与目标系统交互的系统
响应	查找架构中需要修改的位置，进行修改且不会不影响其他功能，对所做的修改进行测试，部署所做的修改
响应度量	根据所影响元素的数量度量的成本、努力、资金；该修改对其他功能或质量属性所造成影响的程度



12.1.3 质量属性场景描述

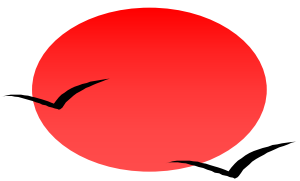
■ 质量属性场景举例：

- 性能场景：①数据传递时延不大于1S，并提供相应的优先级管理。②网站在并发用户数量10万的负载情况下，用户请求的平均响应时间应小于3秒。
- 可用性场景：①系统采用双机热备，主备机必须实时监测对方状态，以便完成系统的实时切换。②主站宕机后，系统能够在10秒内自动切换至备用站点并恢复正常运行。
- 安全性场景：系统应能够防止99%的黑客攻击。
- 可修改性场景：系统完成上线后，少量的外围业务功能和界面的调整与修改不超过10人·月。



12.1.3 质量属性场景描述

- 质量属性场景主要关注可用性、可修改性、性能、可测试性、易用性和安全性等6类质量属性。
 - 可用性质量属性场景所关注的方面包括系统故障发生的频率、出现故障时会发生什么情况、允许系统有多长时间是非正常运行、什么时候可以安全地出现故障、如何防止故障的发生以及发生故障时要求进行哪种通知。
 - 可修改性质量属性场景主要关注系统在改变功能、质量属性时需要付出的成本和难度，可修改性质量属性场景可能发生在系统设计、编译、构建、运行等多种情况和环境下。
 - 性能质量属性场景主要关注系统的响应速度，可以通过效率、响应时间、吞吐量、负载来客观评价性能的好坏。
 - 可测试性质量属性场景主要关注系统测试过程中的效率，发现系统缺陷或故障的难易程度等。
 - 易用性质量属性场景主要关注用户在使用系统时的容易程度，包括系统的学习曲线、完成操作的效率、对系统使用过程的满意程度等。
 - 安全性质量属性场景主要关注系统在安全性方面的要素，衡量系统在向合法用户提供服务的同时，阻止非授权用户使用的能力。

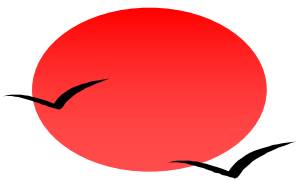


本节练习

- (5)为了精确描述软件系统的质量属性，通常采用_____作为描述质量属性的手段。其中，_____描述在激励到达后所采取的行动。
 - A.质量属性场景
 - B.质量属性环境分析
 - C.质量属性效用树
 - D.质量属性需求
 - A.响应度量
 - B.制品
 - C.响应
 - D.刺激
- 答案：A C
- 解析：为了精确描述软件系统的质量属性，通常采用质量属性场景作为描述质量属性的手段。其中，响应描述在激励到达后所采取的行动。



- 17



12.2 系统架构评估-12.2.1 系统架构评估中的重要概念

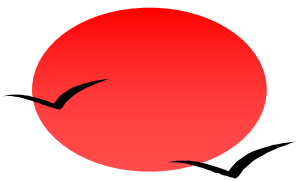
- **系统架构评估**是在对架构分析、评估的基础上，对架构策略的选取进行决策。

- 系统架构评估中的重要概念：

(1) **敏感点和权衡点**。敏感点和权衡点是关键的架构决策。**敏感点**：是指为了实现某种特定的质量属性，一个或多个构件所具有的特性。**权衡点**：是影响多个质量属性的特性，是多个质量属性的敏感点。改变加密级别可能会对安全性和性能产生非常重要的影响。提高加密级别可以提高安全性，但可能要耗费更多的处理时间，影响系统性能。如果某个机密消息的处理有严格的时间延迟要求，则加密级别可能就会成为一个**权衡点**。

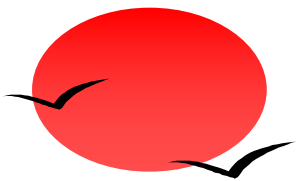
(2) **风险承担者或者称为利益相关人**。系统的架构涉及很多人的利益，这些人都对架构施加各种影响，以保证自己的目标能够实现。例如，风险承担者是软件系统架构师，职责是负责软件架构的质量需求间进行权衡的人，所关心的问题是对其其他风险承担者提出的质量需求的折中和调停。

(3) **场景**。在进行架构评估时，一般首先要精确地得出具体的质量目标，并以之作为判定该架构优劣的标准。为得出这些目标而采用的机制称之为**场景**。场景是从**风险承担者的角度对与系统的交互的简短描述**。在架构评估中，一般采用**刺激、环境和响应**三方面来对场景进行描述。



12.2.1 系统架构评估中的重要概念

- 系统架构评估在**架构设计之后，系统设计之前**，因此与设计、实现、测试都没有关系。评估的目的是**为了评估所采用的架构是否能解决软件系统需求**。
- 系统架构评估的方法有3种：
 - **基于调查问卷或检查表的方法**：类似于需求获取中的**问卷调查**方式，只不过是架构方面的问卷，要求评估人员对领域熟悉。
 - **基于场景的评估方法**：主要方法。它是通过分析软件架构对场景（也就是对系统的使用或修改活动）的支持程度，从而判断该架构对这一场景所代表的质量需求的满足程度。从三个方面对场景进行设计：**刺激**（事件）；**环境**（事件发生的环境）；**响应**（架构响应刺激的过程）。应用在架构权衡分析方法（Architecture Tradeoff Analysis Method, **ATAM**）和软件架构分析方法（Software Architecture Analysis Method, **SAAM**）中。
 - **基于度量的评估方法**：制定一些**定量指标**来度量架构，如代码行数等。要制定质量属性和度量结果之间的映射，要求评估人员对架构熟悉。它是建立在软件架构度量的基础上的，涉及3个基本活动，首先需要建立质量属性和度量之间的映射原则，然后从软件架构文档中获取度量信息，最后根据映射原则分析推导出系统的质量属性。



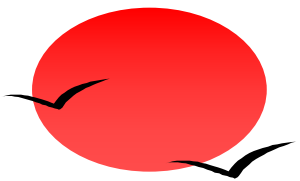
本节练习

- (7)改变加密级别可能会对安全性和性能产生非常重要的影响，因此在软件架构评估中，该设计决策是一个_____。

■ A.敏感点 B.风险点 C.权衡点 D.非风险点

- 答案：C

- 解析：敏感点是一个或多个构件的特性。权衡点是影响多个质量属性的特性，是多个质量属性的敏感点。由题干知，改变加密级别会影响安全性和性能两个质量属性，因此属于权衡点。



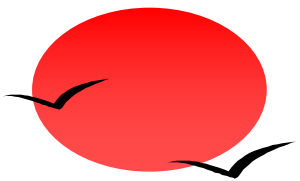
本节练习

- (8)在架构评估中，场景是从_____的角度对与系统交互的描述，一般采用_____三方面来对场景进行描述。

- A. 系统设计者 B. 系统开发者 C. 风险承担者 D. 系统测试者
- A. 刺激源、制品、响应 B. 刺激、制品、响应
- C. 刺激、环境、响应 D. 刺激、制品、环境

- 答案：C C

- 解析：**场景**。在进行架构评估时，一般首先要精确地得出具体的质量目标，并以之作为判定该架构优劣的标准。为得出这些目标而采用的机制称之为场景。场景是从**风险承担者的角度对与系统的交互的简短描述**。在架构评估中，一般采用**刺激、环境和响应**三方面来对场景进行描述。



12.2.2 系统架构评估方法

■ 1. 基于场景的架构分析方法SAAM

SAAM是一种**非功能质量属性**的架构分析方法，是**最早**形成文档并得到广泛使用的软件架构分析方法。

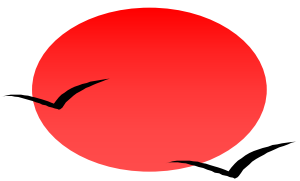
(1) 特定目标。SAAM的目标是对**描述应用程序属性的文档**，**验证基本的架构假设和原则**。

(2) 评估技术。SAAM所使用的评估技术是**场景技术**。场景代表了描述架构属性的基础，描述了各种系统必须支持的活动和可能存在的状态变化。

(3) 质量属性。这一方法的基本特点是把任何形式的质量属性都具体化为场景，但**可修改性**是SAAM分析的主要质量属性。

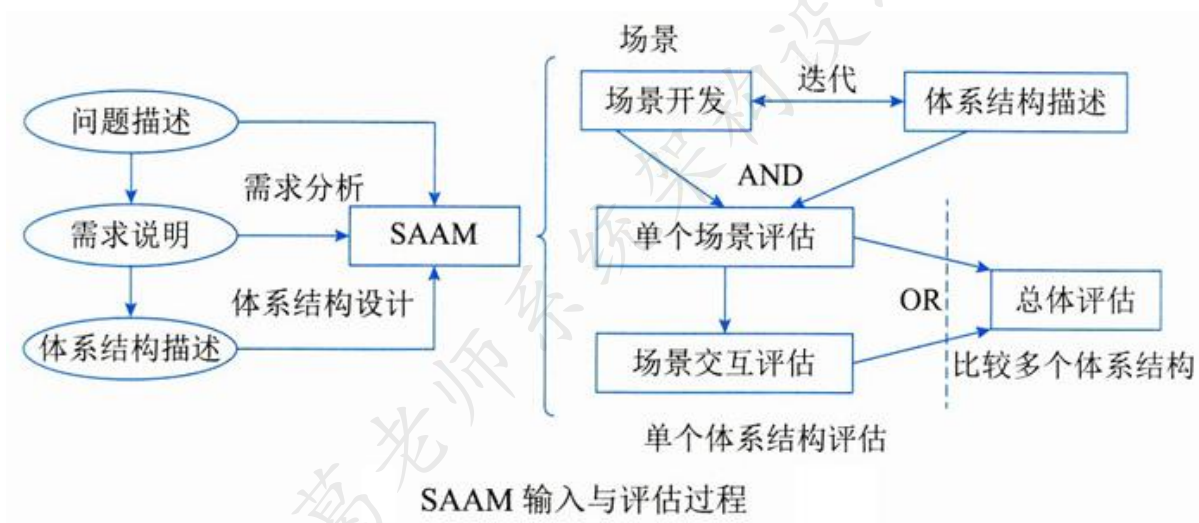
(4) 风险承担者。SAAM协调不同参与者之间感兴趣的共同方面，作为后续决策的基础，达成对架构的共识。

(5) 架构描述。SAAM用于**架构的最后版本**，但早于详细设计。架构的描述形式应当被所有参与者理解。**功能、结构和分配**被定义为描述架构的3个主要方面。



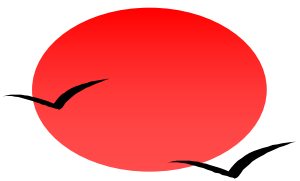
12.2.2 系统架构评估方法

(6) 方法活动。SAAM的主要输入是问题描述、需求声明和架构描述。下图描绘了SAAM分析活动的相关输入及评估过程。SAAM分析评估架构的过程包括5个步骤，即场景开发、架构描述、单个场景评估、场景交互评估和总体评估。



(7) 已有知识库的可重用性：SAAM不考虑这个问题。

(8) 方法验证：SAAM是一种成熟的方法，已被应用到众多系统中，这些系统包括空中交通管制、嵌入式音频系统、WRCS（修正控制系统）、KWIC（根据上下文查找关键词系统）等。



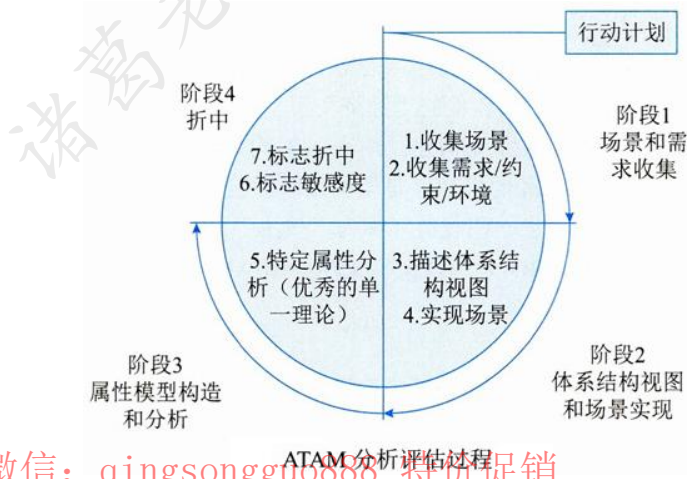
12.2.2 系统架构评估方法

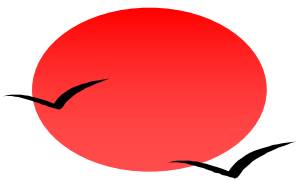
■ 2. 架构权衡分析方法ATAM

架构权衡分析方法ATAM是在SAAM的基础上发展起来的，主要针对性能、安全性、可修改性和可用性，在系统开发之前，对这些质量属性进行评价和折中。

架构权衡分析法ATAM，让架构师明确如何权衡多个质量目标，参与者有评估小组、项目决策者和其他项目相关人。

ATAM被分为四个主要的活动领域，分别是场景和需求收集、体系结构视图和场景实现、属性模型构造和分析、折中。整个评估过程强调以质量属性作为架构评估的核心。





12.2.2 系统架构评估方法

用ATAM方法评估软件架构，其工作分为4个基本阶段，即**演示**、**调查和分析**、**测试和报告**，如图所示。

(1) 阶段1--演示

这是使用ATAM评估软件体系结构的初始阶段。

①介绍ATAM。描述ATAM的评估过程。

②介绍业务驱动因素。着重业务视角，提供有关系统功能、主要利益相关方、业务目标和其他限制等信息。

③介绍要评估的体系结构。侧重可用性以及体系结构的质量要求。

(2) 阶段2--调查和分析

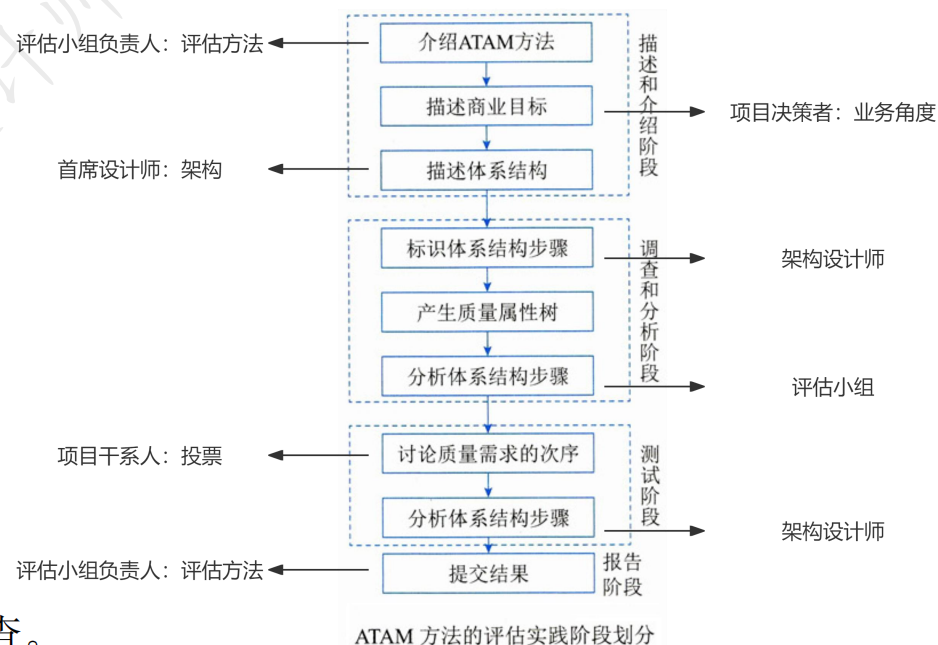
在这个阶段，人们对评估期间需要重点关注的一些关键问题进行彻底调查。

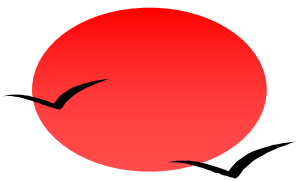
①确定架构方法：涉及能够理解系统关键需求的关键架构方法。

②**生成质量属性效用树：确定最重要的质量属性，并确定有限次序。**

用例、增长、探测

③分析体系结构方法：彻底调查和分析，找出处理相应质量属性架构的方法。包括4个主要阶段：调查架构方法->创建分析问题->分析问题的答案->找出风险、非风险、敏感点和权衡点。





12.2.2 系统架构评估方法

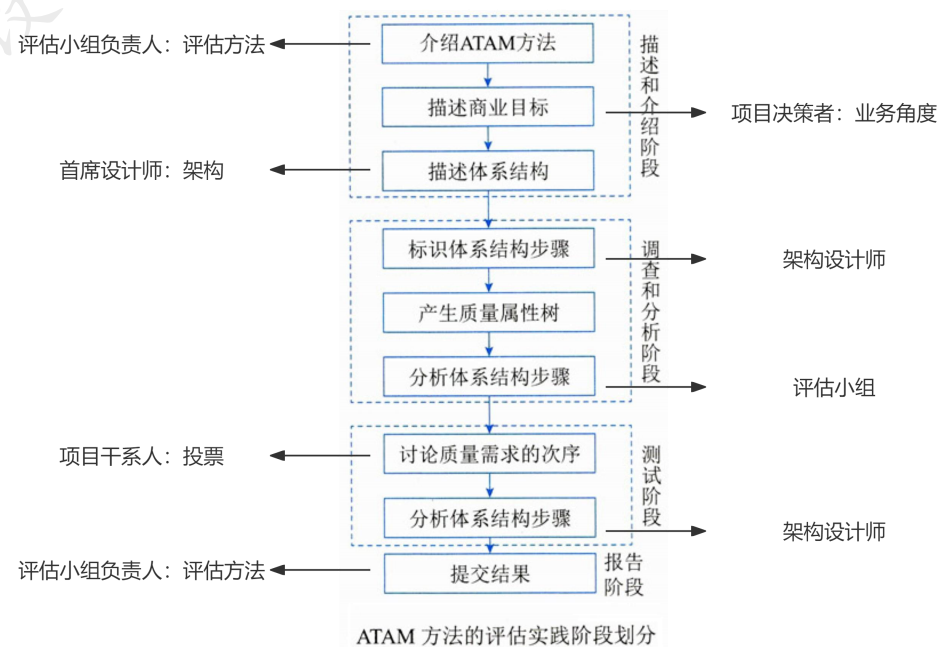
(3) 阶段3--测试

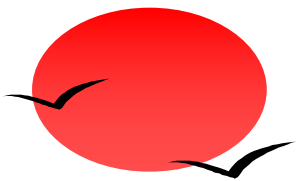
①头脑风暴和优先场景：将头脑风暴的优先列表与生成质量属性效用树中所获取的优先方案进行比较。

②分析架构方法。

(4) 阶段4--报告ATAM

提供评估期间收集的所有信息，呈现给利益相关者。

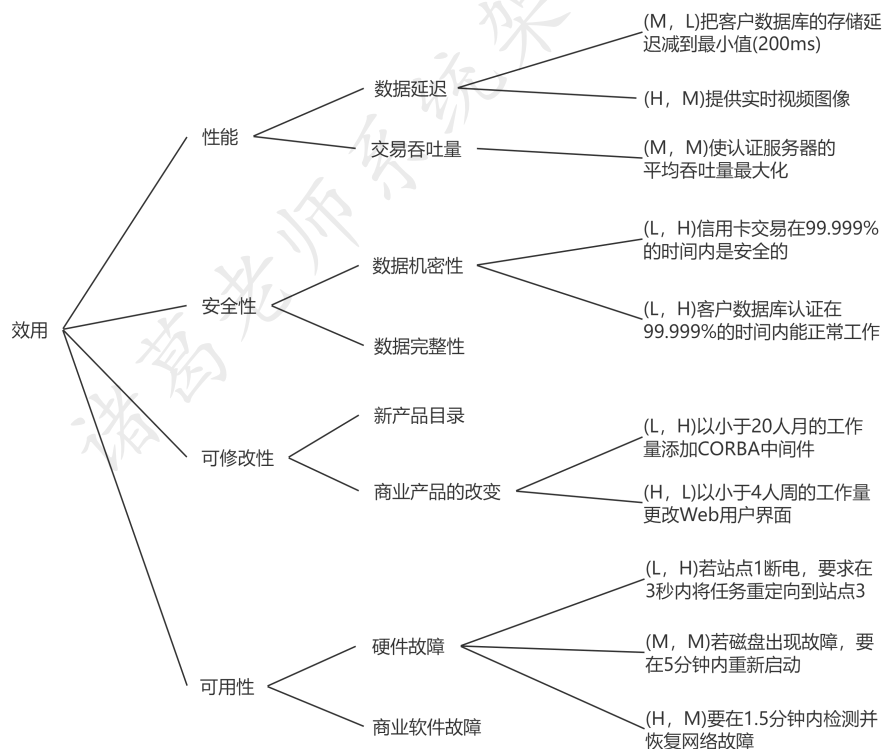


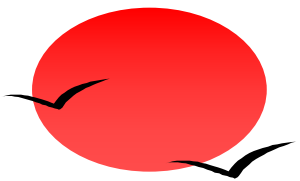


12.2.2 系统架构评估方法

■ 质量属性效用树

ATAM方法采用**效用树**（Utility tree）这一工具来对质量属性进行分类和优先级排序。**效用树的结构包括：树根--质量属性--属性分类--质量属性场景（叶子节点）**。需要注意的是，ATAM主要关注4类质量属性：**性能、安全性、可修改性和可用性**，这是因为这4个质量属性是利益相关者最为关心的。



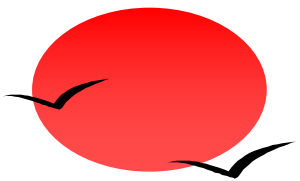


12.2.2 系统架构评估方法

■ 3. 成本效益分析法CBAM

成本效益分析法CBAM是在ATAM上构建，用来对架构设计决策的成本和收益进行建模，让决策者根据投资回报率ROI来选择合适的架构，在ATAM确定质量合理的基础上，再对效益进行分析。有以下8个步骤：

- (1) 整理场景（确定场景，并确定优先级，选择优先级最高的1/3场景进行分析）；
- (2) 对场景进行细化（对每个场景详细分析，确定最好、最坏的情况）；
- (3) 确定场景的优先级（项目干系人对场景投票，根据投票结果生成场景的权值）；
- (4) 分配效用（对场景响应级别确定效用表，建立策略、场景、响应级别的表格）；
- (5) 形成“策略-场景-响应级别”的对应关系；
- (6) 使用内插法确定期望的质量属性响应级别的效用（根据效用表确定所对应的具体场景的效用表）；
- (7) 计算各架构策略的总收益；
- (8) 根据受成本限制影响的投资回报率ROI选择架构策略（估算成本，用上一步的收益减去成本，计算ROI并排序，从而选择收益最高的架构策略）。



12.2.2 系统架构评估方法

■ 4. 其他评估方法（仅了解）

1.SAEM方法。将软件架构看作一个最终产品以及设计过程中的一个中间产品，从外部质量属性和内部质量属性两个角度来阐述它的评估模型，旨在为软件架构的质量评估创建一个基础框架。

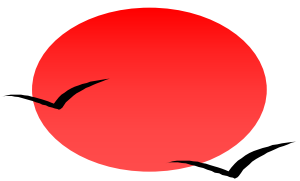
外部属性指用户定义的质量属性，而内部属性指开发者决定的质量属性。该软件架构评估模型包含以下几个流程。

（1）对待评估的质量属性进行规约建模。

（2）为外部和内部的质量属性创建度量准则，先从评估目的（如软件架构比较、最终产品的质量预测），评估角度（如开发者、用户、维护者），评估环境（架构作为最终产品或设计中间产品）出发来定义架构评估的目标，再根据目标相关的属性来提出问题，然后回答每个问题并提出相应的度量准则。

（3）评估质量属性，包括数据收集、度量和结果分析3个活动。

2.SAABNet方法。是一种用来表达和使用定性知识以辅助架构的定性评估。该方法来源于人工智能，允许不确定、不完整知识的推理。该方法使用BBN来表示和使用开发过程中的知识，包含定性和定量的描述，其中定性的描述是所有结点的图，定量的描述是每个结点状态相关的条件概率。其中的变量可分为3类，即架构质量属性变量（如可维护性、灵活性等）、质量属性的度量准则变量（如容错性、响应性等）和架构特征变量（如继承深度、编程语言等），高层抽象的质量属性变量分解为低层抽象的度量准则变量，度量准则变量则分解为更低层抽象的架构特征变量。



12.2.2 系统架构评估方法

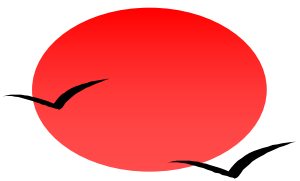
3.SACMM方法。是一种**软件架构修改**的度量方法。

4.SASAM方法。通过对**预期架构**（架构设计阶段的相关描述材料）和**实际架构**（源代码中执行的架构）**进行映射和比较**来静态地评估软件架构。

5.ALRRRA方法。是一种**软件架构可靠性风险评估方法**，该方法使用动态复杂度准则和动态耦合度准则来定义组件和连接件的复杂性因素，其中，动态复杂度准则在某个场景的执行中分析组件的动态行为来度量组件的复杂性，动态耦合度准则在某个场景的执行中分析连接件的消息传递协议来度量连接件的复杂性。利用失效模式和影响分析（FMEA）技术。

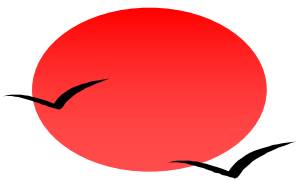
6.AHP方法（层次分析法）。是**对定性问题进行定量分析**的一种简便、灵活而又实用的多准则决策方法。AHP方法的特点是把复杂问题中的各种因素通过划分为相联系的有序层次使之条理化，并在一般情况下通过两两对比，根据一定客观现实的主观判断结构，把专家意见和分析者的客观判断结果直接、有效地结合起来，将一定层次上元素的某些重要性进行定量描述，之后利用数学方法计算反映每一层次元素的相对重要性次序的权值，并最后通过所有层次之间的总排序计算所有元素的相对权重及对权重进行排序。

7.COSMIC+UML方法。基于**度量模型来评估软件架构可维护性**的方法。针对不同表达方式的软件架构，采用统一的软件度量COSMIC方法来进行度量和评估。该方法主要是为了辅助分析软件架构的演化方案是否可行，并在开源软件DCMMS的软件架构UML组件图上得以验证。



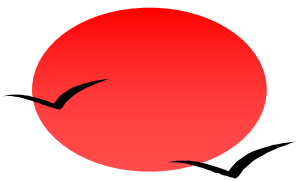
本节练习

- (9)在ATAM评估方法设计之初，其主要关注的4种质量属性，分别为_____。
 - A.性能、安全性、可修改性和可测试性
 - B.性能、安全性、可修改性和可用性
 - C.性能、可修改性、可用性和可测试性
 - D.安全性、可测试性、可用性和可测试性
- 答案：B
- 解析：ATAM方法采用**效用树**（Utility tree）这一工具来对质量属性进行分类和优先级排序。**效用树的结构包括：树根--质量属性--属性分类--质量属性场景（叶子节点）**。需要注意的是，ATAM主要关注4类质量属性：**性能、安全性、可修改性和可用性**，这是因为这4个质量属性是利益相关者最为关心的。



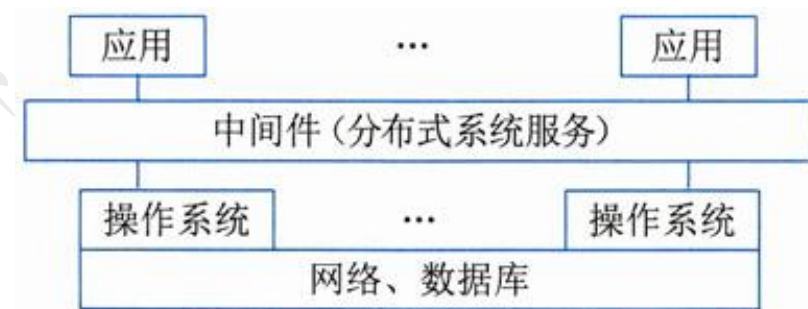
本节练习

- (10)效用树是采用架构权衡分析方法（Architecture Trade off Analysis Method, ATAM）进行架构评估的工具之一，其树形结构从根部到叶子节点依次为_____。
 - A.树根、属性分类、优先级、质量属性场景
 - B.树根、质量属性、属性分类、质量属性场景
 - C.树根、优先级、质量属性、质量属性场景
 - D.树根、质量属性、属性分类、优先级
- 答案：B
- 解析：ATAM方法采用**效用树**（Utility tree）这一工具来对质量属性进行分类和优先级排序。**效用树的结构包括：树根--质量属性--属性分类--质量属性场景（叶子节点）**。需要注意的是，ATAM主要关注4类质量属性：**性能、安全性、可修改性和可用性**，这是因为这4个质量属性是利益相关者最为关心的。



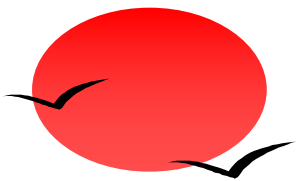
中间件

- 中间件是指在一个分布式系统环境中处于操作系统和应用程序之间的系统级软件，可以在不同的技术之间共享资源，将不同的操作系统、数据库、异构的网络环境以及若干应用结合成一个有机的协同工作整体。



中间件图示

- 中间件位于客户机/服务器的操作系统之上，管理计算机资源和网络通信，有如下特点：
 - (1) 中间件是一类软件，而非一种软件；
 - (2) 中间件不仅仅实现互连，还要实现应用之间的互操作；
 - (3) 中间件是基于分布式处理的软件，最突出的特点是其网络通信功能。
- 中间件的任务是使应用程序开发变得更容易，通过提供统一的程序抽象，隐藏异构系统和分布式系统下低级别编程的复杂度。

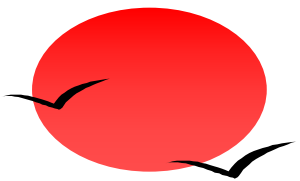


中间件

- 中间件提供的支持通常包括两方面：

- **交互支持**：协调系统中不同组件之间的通信和数据交换。中间件可以提供消息队列、远程过程调用（RPC）、对象请求代理（ORB）等机制，以实现分布式环境中的进程间通信（IPC）。这些机制使得应用程序不必关心底层网络细节，能够更专注于业务逻辑。

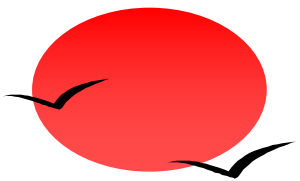
- **提供公共服务**：中间件提供对服务的可复用的实现，如事务管理、安全服务、命名和目录服务、持久化服务、负载均衡、故障恢复和容错能力等。这些服务有助于解决分布式系统中常见的问题，如一致性、可用性和伸缩性。



中间件

■ 中间件的功能：

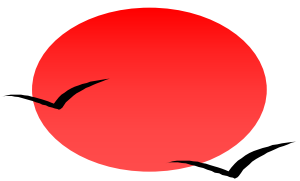
- 负责客户机与服务器之间的**连接和通信**，以及客户机与应用层之间的高效率通信机制。
- 提供应用层不同服务之间的**互操作机制**，以及应用层与数据库之间的连接和控制机制。
- 提供多层架构的应用开发和运行的平台，以及应用开发框架，支持模块化的应用开发。
- 屏蔽硬件、操作系统、网络和数据库的差异。
- 提供应用的**负载均衡和高可用性、安全机制与管理功能，以及交易管理机制**，保证交易的一致性。
- 提供一组通用的服务去执行不同的功能，避免重复的工作和使应用之间可以协作。



中间件

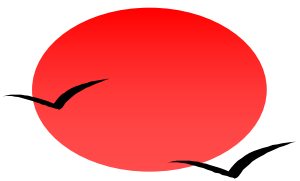
■ 中间件的分类：

- (1) 通信处理（消息）中间件，保证系统能在不同平台之间通信，利用消息传递机制实现分布式系统中可靠的、高效的、实时的跨平台数据传输。例如IBM的MQSeries。
- (2) 事务处理（交易）中间件，实现协调处理顺序、监视和调度、负载均衡等功能。例如BEA的Tuxedo。
- (3) 数据存取管理中间件，为不同种类数据的读写和加解密提供统一的接口。例如Windows平台的ODBC和Java平台的JDBC等。
- (4) Web服务器中间件，提供Web程序执行的运行时容器。例如Tomcat、JBOSS等。
- (5) 安全中间件，用中间件屏蔽操作系统的缺陷，提升安全等级。例如Kerberos、SSL/TLS。
- (6) 跨平台和架构的中间件，用于开发大型应用软件。例如CORBA、JavaBeans、COM+模型。
- (7) 专用平台中间件，为解决特定应用领域的开发设计问题提供构件库。例如Android SDK、iOS SDK。
- (8) 网络中间件，包括网管、接入、网络测试、虚拟社区和虚拟缓冲等，也是当前最热门的研发项目。例如TCP/IP协议栈、HTTP服务器。



本节练习

- (11)在分布式系统中，中间件通常提供两种不同类型的支持，即_____。
 - A.数据支持和交互支持
 - B.交互支持和提供公共服务
 - C.安全支持和提供公共服务
 - D.数据支持和提供公共服务
- 答案：B
- 解析：中间件是独立的系统级软件，连接操作系统层和应用程序层，将不同操作系统提供应用的接口标准化，协议统一化，屏蔽具体操作的细节，中间件提供的支持通常包括两方面：
 - 交互支持：协调系统中不同组件之间的通信和数据交换。中间件可以提供消息队列、远程过程调用（RPC）、对象请求代理（ORB）等机制，以实现分布式环境中的进程间通信（IPC）。这些机制使得应用程序不必关心底层网络细节，能够更专注于业务逻辑。
 - 提供公共服务：中间件提供对服务的可复用的实现，如事务管理、安全服务、命名和目录服务、持久化服务、负载均衡、故障恢复和容错能力等。这些服务有助于解决分布式系统中常见的问题，如一致性、可用性和伸缩性。



感谢

THANKS

系统架构设计师QQ群：231352210

软件设计师QQ群：1169209218

诸葛老师QQ：362842353